

DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT(25CS1302E)

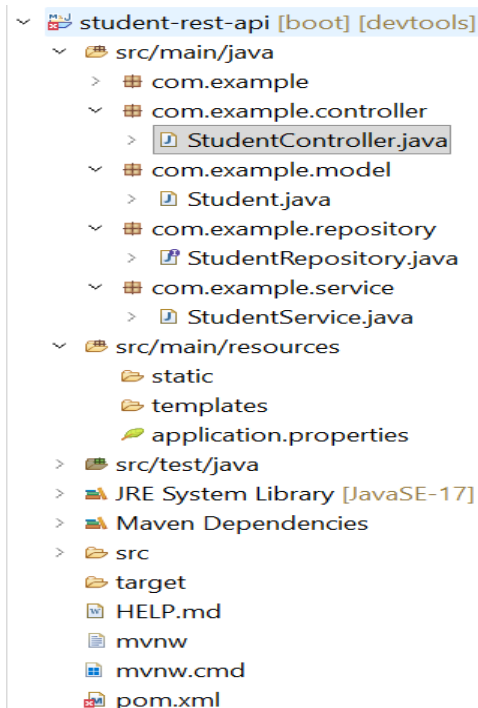
LAB MANUAL

1. A working REST API running on day one — students issue HTTP requests with curl and Postman and observe the round trip before any internals. The canonical backend stack: HTTP server → routing/middleware → service layer → data-access layer → ORM/JDBC → SQL → storage engine → filesystem. REST as a discipline: resources, HTTP verbs, status codes, idempotence, statelessness; the contract a backend exposes to its clients. Spring Boot at intro level: the `@RestController`, `@Service`, `@Repository` layering; dependency injection at use-site, not theory.

Case Study: Student Management REST API

- Develop a REST API to manage students in a college. The API should allow clients (Postman/cURL) to perform CRUD operations.

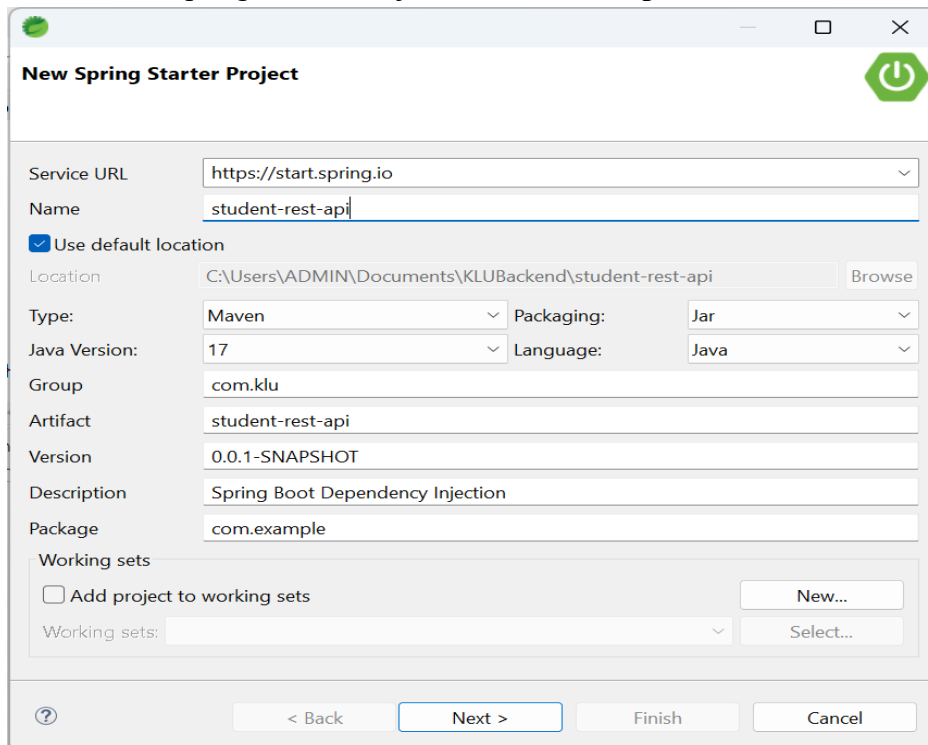
Project Structure:



Step 1:

- Open SpringBoot workspace

- Create New Spring Starter Project **student-rest-api**



New Spring Starter Project

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

Description:

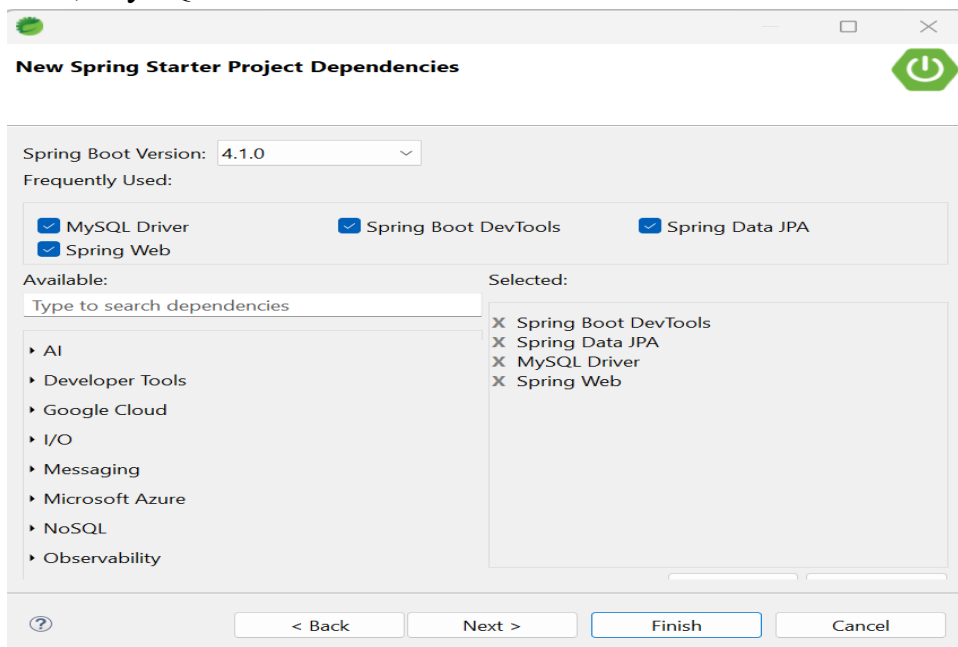
Package:

Working sets

☐ Add project to working sets

Working sets:

- Next, select dependencies Spring Boot DevTools, Spring Data JPA, Spring Web, MySQL Driver.



New Spring Starter Project Dependencies

Spring Boot Version:

Frequently Used:

☒ MySQL Driver ☒ Spring Boot DevTools ☒ Spring Data JPA

☒ Spring Web

Available:

Type to search dependencies

Available categories:

- AI
- Developer Tools
- Google Cloud
- I/O
- Messaging
- Microsoft Azure
- NoSQL
- Observability

Selected:

- X Spring Boot DevTools
- X Spring Data JPA
- X MySQL Driver
- X Spring Web

- Click on Next and Finish.

Step 2:

Add Dependencies in pom.xml file and save the file

```

<?xml version="1.0" encoding="UTF-8"?>

<project                                xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <parent>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-parent</artifactId>

    <version>4.1.0</version>

    <relativePath/> <!-- lookup parent from repository -->

  </parent>

  <groupId>com.klu</groupId>

  <artifactId>student-rest-api</artifactId>

  <version>0.0.1-SNAPSHOT</version>

  <name>student-rest-api</name>

  <description>Spring Boot Dependency Injection</description>

  <url/>

  <licenses>

    <license/>

  </licenses>

  <developers>

    <developer/>

  </developers>

  <scm>

    <connection/>

    <developerConnection/>

    <tag/>

```

```
<url/>

</scm>

<properties>
    <java.version>17</java.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
```

```

        <artifactId>spring-boot-starter-data-jpa-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
</project>

```

Step 3:

In application.properties file,

spring.application.name=student-rest-api

spring.datasource.url=jdbc:mysql://localhost:3306/studentdb

spring.datasource.username=root

spring.datasource.password=root

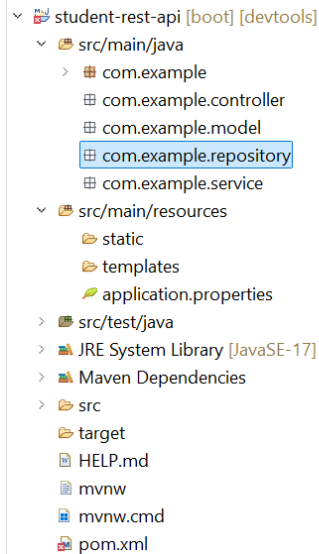
spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

server.port=8085

Step 4:

Create Packages, Model, Controller, Service, Repository in src/main/java



Step 5:

In **com.example.model** Layer, create the class file **student.java** and save it

```
        this.name=name;
    }

    public String getDepartment() {
        return department;
    }

    public void setDepartment(String department) {
        this.department=department;
    }

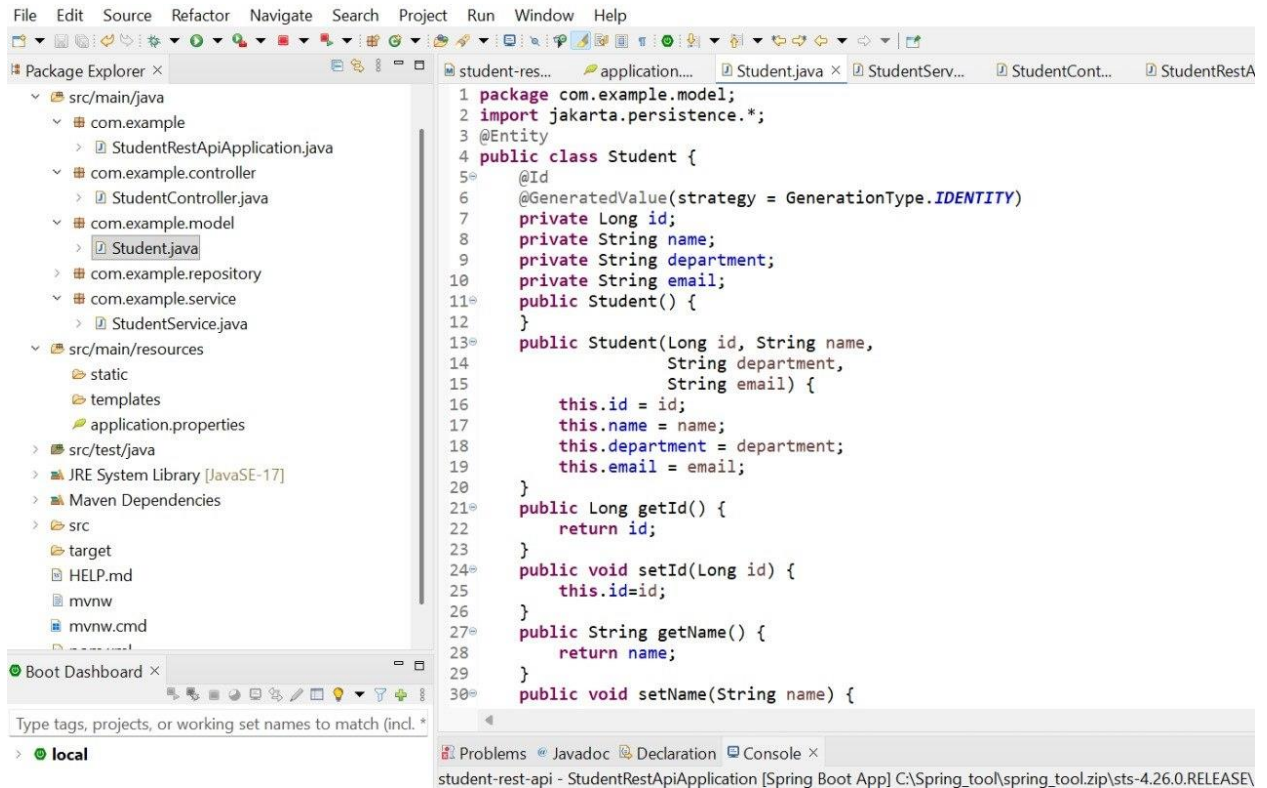
    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
```

```

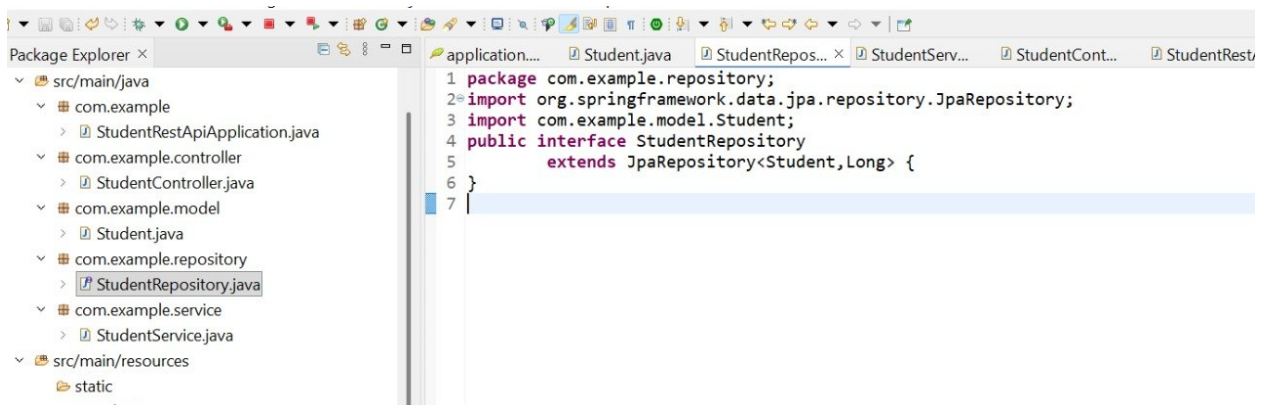
        this.email=email;
    }
}

```



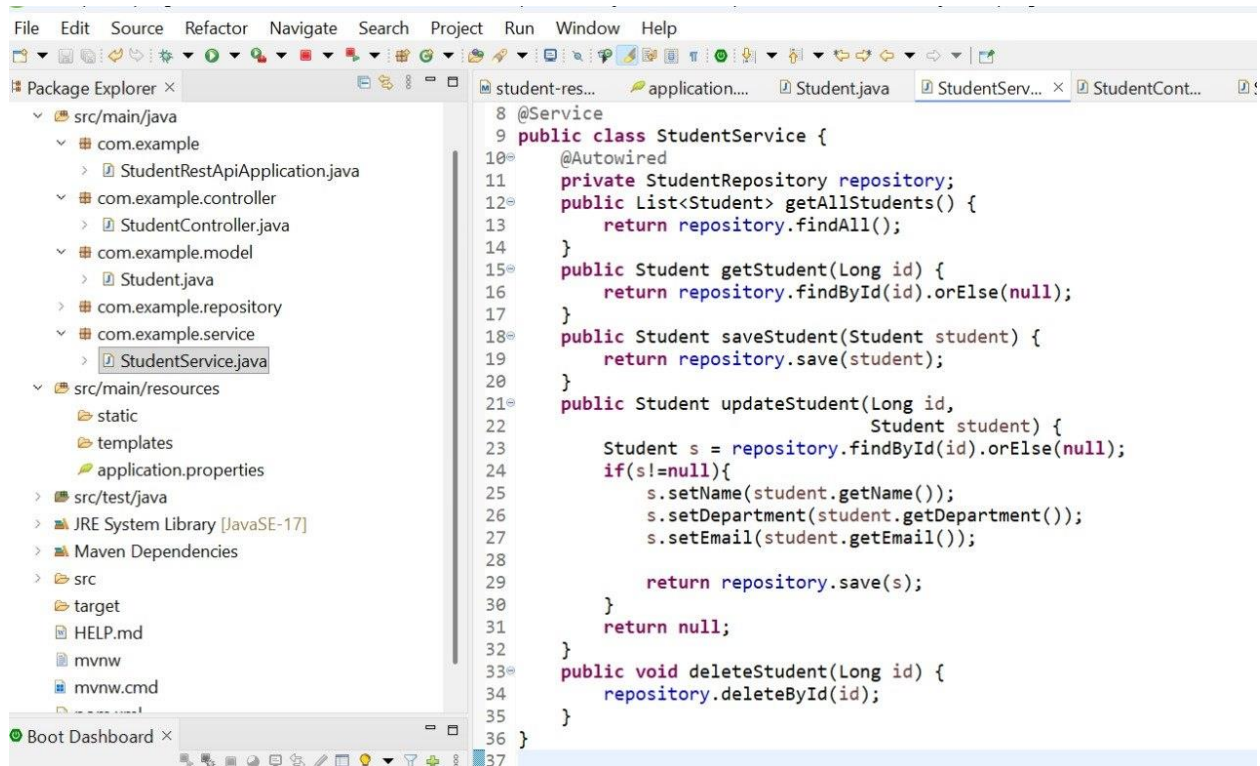
Step 6:

Create class file StudentRepository.java in com.example.Repository Layer and save it.



Step 7:

Create class file **StudentService.java** in **com.example.Service** Layer and save it.



Step 8:

Create class file **StudentController.java** in **com.example.Controller** Layer and save it.

```
package com.example.controller;
```

```
import java.util.List;
```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.http.ResponseEntity;
```

```
import org.springframework.web.bind.annotation.*;
```

```
import com.example.model.Student;
```

```
import com.example.service.StudentService;
```

```
@RestController
```

```
@RequestMapping("/students")
public class StudentController {

    @Autowired
    private StudentService service;

    @GetMapping
    public List<Student> getAllStudents() {
        return service.getAllStudents();
    }

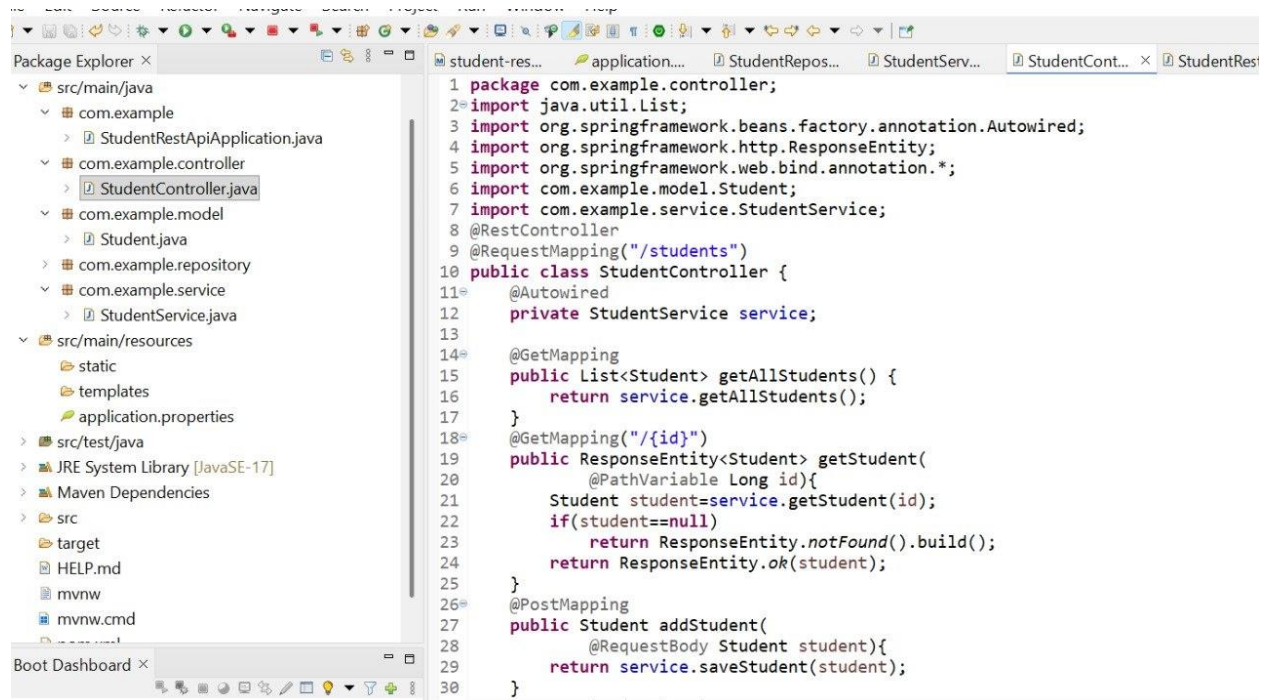
    @GetMapping("/{id}")
    public ResponseEntity<Student> getStudent(
        @PathVariable Long id){
        Student student=service.getStudent(id);
        if(student==null)
            return ResponseEntity.notFound().build();
        return ResponseEntity.ok(student);
    }

    @PostMapping
    public Student addStudent(
        @RequestBody Student student){
        return service.saveStudent(student);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Student> updateStudent(
```

```
        @PathVariable Long id,
        @RequestBody Student student){
    Student updated=service.updateStudent(id,student);
    if(updated==null)
        return ResponseEntity.notFound().build();

    return ResponseEntity.ok(updated);
}
    @DeleteMapping("/{id}")
    public ResponseEntity<String> deleteStudent(
        @PathVariable Long id){
        service.deleteStudent(id);
        return ResponseEntity.ok("Deleted Successfully");
    }
}
```



Step 9:

- Start MySQL Server
- Create Database StudentDb;

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

```
mysql> create database studentdb;  
Query OK, 1 row affected (0.04 sec)
```

```
mysql> use studentdb;
```

Database changed

```
mysql> show tables;
```

```
+-----+  
| Tables_in_studentdb |  
+-----+  
| student              |  
+-----+  
1 row in set (0.04 sec)
```

```
mysql> select * from student;
```

```
+----+-----+-----+-----+  
| id | department | email          | name |  
+----+-----+-----+-----+  
|  1 | CSE         | ram@gmail.com  | Ram  |  
+----+-----+-----+-----+  
1 row in set (0.00 sec)
```

```
mysql> |
```

Step 10:

Run as SpringBoot App

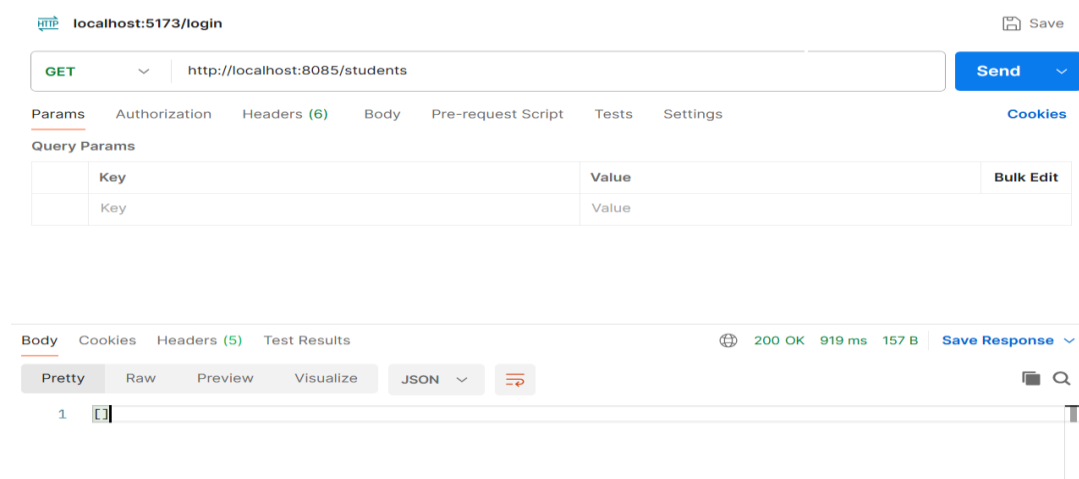
```
Console x
student-rest-api - StudentRestApiApplication [Spring Boot App] C:\Program Files\Java\jdk-21\bin\javaw.exe (Jul 27, 2026, 10:30:08AM) [pid: 323032]
org.hibernate.orm.core : HHH000001: Hibernate ORM core version 7.4.1.Final
o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup: ignoring JPA class transformer
com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection com.mysql.cj.jdbc.ConnectionImpl@724ec12a
com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
org.hibernate.orm.connections.pooling : HHH10001005: Database info:

org.hibernate.orm.core : HHH0000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to e
j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
o.s.d.j.r.query.QueryEnhancerFactories : Hibernate is in classpath; If applicable, HQL parser will be used.
JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be p
o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8085 (http) with context path '/'
com.example.StudentRestApiApplication : Started StudentRestApiApplication in 6.751 seconds (process running for 7.701)
```

Step 11:

Open Postman app,

Testing GET method with Postman



Since there are no data in the table, it returned empty braces.

Testing POST method with Postman

localhost:5173/login

POST http://localhost:8085/students

Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "name": "Ram",
3   "department": "CSE",
4   "email": "ram@gmail.com"
5 }
```

Body Cookies Headers (5) Test Results 200 OK 435 ms 220 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 1,
3   "name": "Ram",
4   "department": "CSE",
5   "email": "ram@gmail.com"
6 }
```

Now, inserted on row into the table using POST method. Test it with GET method to retrieve the data.

localhost:5173/login

GET http://localhost:8085/students

Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```
1 [
2   {
3     "id": 1,
4     "name": "Ram",
5     "department": "CSE",
6     "email": "ram@gmail.com"
7   }
8 ]
```

Body Cookies Headers (5) Test Results 200 OK 19 ms 222 B Save Response

Pretty Raw Preview Visualize JSON

Step 12:

Now test it with MySQL Server

- Use Studentdb;
- Show Tables;

```
mysql> use studentdb;
Database changed
mysql> show tables;
+-----+
| Tables_in_studentdb |
+-----+
| student              |
+-----+
1 row in set (0.02 sec)
```

- Now verify, whether the row inserted using POST method in POSTMAN is visible in Database.

```
mysql> use studentdb;
Database changed
mysql> show tables;
+-----+
| Tables_in_studentdb |
+-----+
| student              |
+-----+
1 row in set (0.02 sec)

mysql> select * from student;
+----+-----+-----+-----+
| id | department | email          | name |
+----+-----+-----+-----+
| 1  | CSE        | ram@gmail.com  | Ram  |
+----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

Likewise test other methods PUT and DELETE.

Step 13:

Testing with cURL:

cURL stands for Client URL. It is a command-line tool used to send HTTP requests to a web server or REST API. It allows developers to test APIs without using a graphical interface like Postman.

Why use cURL?

- Test REST APIs from the terminal.
- Send HTTP requests (GET, POST, PUT, DELETE).
- Debug backend services.
- Automate API testing in scripts.

Run in Command Prompt:

```
Command Prompt
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>curl http://localhost:8085/students
[{"id":1,"name":"Ram","department":"CSE","email":"ram@gmail.com"}]
C:\Users\ADMIN>
```

Create a student:

curl -X POST http://localhost:8085/students -H "Content-Type: application/json" -d '{"name":"Ramya","department":"CSE","email":"ramya@gmail.com"}'

```
C:\Users\ADMIN>curl -X POST http://localhost:8085/students -H "Content-Type: application/json" -d '{"name":"Ramya","department":"CSE","email":"ramya@gmail.com"}'
{"id":3,"name":"Ramya","department":"CSE","email":"ramya@gmail.com"}
C:\Users\ADMIN>
```

Check all the students details:

```
C:\Users\ADMIN>curl http://localhost:8085/students
[{"id":1,"name":"Ram","department":"CSE","email":"ram@gmail.com"}, {"id":2,"name":"Ram","department":"CSE","email":"ram@gmail.com"}, {"id":3,"name":"Ramya","department":"CSE","email":"ramya@gmail.com"}]
C:\Users\ADMIN>
```